مكتبة لالا

دليل مطوّرة الباك إند

التعديلات والإصلاحات التي تمت على كود Laravel، والراوتات الجديدة، والميزات المطلوب بناؤها مع عقود الـ API، وقائمة متابعة لخطة المشروع.

الإصدار الأول · 7 تشرين الثاني 2026

محتويات الدليل

1. كيف تقرئين هذا الدليل
2. تشغيل المشروع بعد git pull
3. الإصلاحات التي تمت على الكود الموجود (مع السبب والمكان)
4. التوابع والراوتات الجديدة التي أُضيفت
5. جدول كل الراوتات الحالية
6. الميزات المطلوب بناؤها (الراوتات والتوابع الناقصة)
7. قائمة المتابعة وخطة بناء المشروع

1. كيف تقرئين هذا الدليل

هدف المشروع أن تتعلمي مفاهيم Laravel على منصة حقيقية. لذلك كل تعديل على كودك محفوظ داخل الملف نفسه بهذا الشكل حتى تقارني بين القديم والجديد:

```
/*
==========================================================
PREVIOUS IMPLEMENTATION - KEPT FOR REVIEW
Reason: ...
==========================================================
*/
// old code, commented out
/* UPDATED IMPLEMENTATION */
new code
```

والإضافات الجديدة تبدأ بتعليق `ADDED`. الاستثناء الوحيد: الـ Seeders وملف Docker، لأن الكود القديم فيهما كان يحتوي كلمات مرور، فلم نحتفظ به كتعليق (يمكن رؤيته عبر `git log -p`).

ملفات مرجعية داخل المستودع:

- `docs/BACKEND_CHANGES.md`: شرح تفصيلي لكل إصلاح والمفهوم الذي يعلّمه.
- `docs/BACKEND_PENDING.md`: العقود الكاملة للميزات المطلوبة (أشكال الطلبات والاستجابات بالتفصيل).
- `docs/PROJECT_STATUS.md`: جدول حالة كل ميزة.
- `frontend/src/lib/pending/examples.ts`: بيانات مثال بالشكل الدقيق الذي يجب أن يعيده كل Endpoint. افتحيه بجانب الـ Controller وأنتِ تبنين.
- `DOCKER_SETUP.md`: كل أوامر Docker وحل المشاكل.

هيكلية المستودع:

المسار	المحتوى
`backend/`	كل كود Laravel (app, routes, database, config, storage…) وملف `backend/.env`
`frontend/`	كل كود Next.js
`docker/`	إعدادات nginx و php المشتركة
`docs/`	الوثائق المشتركة
`Dockerfile` · `docker-compose.yml`	تعريف صورة PHP وكل الخدمات
`.env`	متغيرات Docker: المنافذ وكلمات مرور قاعدة البيانات

> كل مسارات ملفات Laravel في هذا الدليل نسبية إلى مجلد `backend/`. مثلًا `routes/api.php` تعني `backend/routes/api.php`، و `app/Http/Controllers` تعني `backend/app/Http/Controllers`.

2. تشغيل المشروع بعد git pull

المطلوب على جهازك فقط: Git و Docker Desktop لا حاجة لتثبيت PHP أو Composer أو MySQL أو Node.

الخطوة 1: جلب آخر نسخة من main

```
cd lalalibrary
git checkout main
git pull origin main
```

> إذا كانت عندك تعديلات محلية غير محفوظة، احفظيها أولًا بـ `git stash` ثم أرجعيها بعد السحب بـ `git stash pop`. وإذا ظهر تعارض (conflict) في ملف، فالنسخة الجديدة في main هي المرجع.
>
> كود Laravel انتقل إلى مجلد `backend/`. إذا كان عندك ملف `.env` قديم في الجذر فانسخي منه ما تحتاجينه ثم أنشئي الملفين الجديدين كما يلي.

الخطوة 2: ملفا البيئة

يوجد ملفان: `.env` في الجذر لـ Docker (كلمات مرور MySQL والمنافذ)، و `backend/.env` لإعدادات Laravel. داخل Docker تُمرر بيانات قاعدة البيانات من ملف الجذر إلى Laravel، فتكتبين كلمة المرور مرة واحدة فقط.

```
cp .env.example .env
cp backend/.env.example backend/.env
```

ثم املئي على الأقل:

المتغير	ماذا تضعين فيه
`DB_PASSWORD`	في `.env`: كلمة مرور مستخدم قاعدة البيانات (أي كلمة قوية من اختيارك)
`DB_ROOT_PASSWORD`	في `.env`: كلمة مرور root لـ MySQL
`SEED_ADMIN_EMAIL`	في `backend/.env`: بريد حساب الأدمن الأول (الافتراضي admin@lalalibrary.test)
`SEED_ADMIN_PASSWORD`	في `backend/.env`: كلمة مرور الأدمن. إذا تركتِها فارغة تُولَّد كلمة عشوائية وتُطبع عند تشغيل الـ Seeder
`SEED_USER_PASSWORD`	في `backend/.env`: كلمة مرور الحسابات التجريبية (الافتراضي password)

الخطوة 3: التشغيل

```
docker compose up -d --build
```

في أول مرة تُثبَّت حزم Composer تلقائيًا (بضع دقائق). تابعي التقدم بالأمر التالي، وعند ظهور `ready to handle connections` اضغطي Ctrl+C:

```
docker compose logs -f app
```

ثم أنشئي الجداول والبيانات الأولية:

```
docker compose exec app php artisan migrate --seed
```

> قاعدة البيانات الآن في Volume جديد اسمه db_data، لذلك ستبدأ فارغة في أول تشغيل، وهذا طبيعي. أمر `migrate --seed` ينشئ الجداول ويضيف الأدوار والأدمن والحسابات التجريبية والتصنيفات.

الخطوة 4: الروابط

الخدمة	الرابط
الواجهة (Next.js)	`http://localhost:3000` (أول تشغيل يأخذ دقيقتين لتثبيت حزم npm)

الخدمة	الرابط
الـ API (Laravel)	`http://localhost:8000/api`
فحص Laravel	`http://localhost:8000/up`
قاعدة البيانات من DBeaver أو غيره	`127.0.0.1:3306` بمستخدم `lalalibrary` وكلمة `DB_PASSWORD`

الحسابات التجريبية

الحساب	البريد	كلمة المرور
أدمن	قيمة `SEED_ADMIN_EMAIL`	قيمة `SEED_ADMIN_PASSWORD`
مستخدمون	`dalaa@example.com` · `sdra@example.com` · `test@example.com`	قيمة `SEED_USER_PASSWORD`

أوامر ستحتاجينها كثيرًا

المهمة	الأمر
إنشاء Controller	`docker compose exec app php artisan make:controller LibraryController`
إنشاء Model مع Migration	`docker compose exec app php artisan make:model Review -m`
إنشاء Form Request	`docker compose exec app php artisan make:request StoreReviewRequest`
إنشاء API Resource	`docker compose exec app php artisan make:resource ReviewResource`
تشغيل Migrations الجديدة	`docker compose exec app php artisan migrate`
عرض الراوتات	`docker compose exec app php artisan route:list --path=api`
Tinker	`docker compose exec app php artisan tinker`
السجلات	`docker compose logs -f app`
إيقاف المشروع (البيانات تبقى)	`docker compose down`

انتبهي: `php artisan migrate:fresh` يحذف كل الجداول، و `docker compose down -v` يحذف قاعدة البيانات كاملة. استخدميهما فقط إن أردتِ البدء من الصفر.

الاختبار من Postman

- أضيفي دائمًا الهيدر `Accept: application/json`.
- سجّلي الدخول بـ `POST /api/login` وانسخي `access_token`، ثم أرسليه في الطلبات المحمية: `Authorization: Bearer <token>`.

وضع المعاينة في الواجهة

في Docker يعمل وضع المعاينة للمطورين (`SHOW_API_CONTRACTS=true`). أي صفحة لميزة لم تبنيها بعد تعرض الواجهة النهائية ببيانات مثال داخل إطار متقطع، وفوقها العقد المطلوب منك. جرّبي مثلًا: `/ar/store` و `/ar/account/library` و `/ar/admin/orders`.

3. الإصلاحات التي تمت على الكود الموجود

هذه أخطاء أو نواقص وُجدت في الكود الذي كتبتِه، مع سبب كل إصلاح ومكانه. الشرح الأطول لكل بند في `docs/BACKEND_CHANGES..md`

3.1 المستخدمون والمصادقة

#	المشكلة	لماذا هي مشكلة	الإصلاح	المكان
1	قاعدة التحقق `unique: user,email`	اسم الجدول `users`، فكل طلب يحتوي email ينتهي بخطأ 500	`unique:users,email,{id}`	`UserController@updateUser`
2	التحقق من `role` واستخدام `roles`	إرسال roles يفشل دائمًا، وإرسال role يجعل `sync(null)` تحذف كل الأدوار بما فيها الأدمن	التحقق من `roles` نفسها، ومنع الأدمن من إزالة دور الأدمن عن نفسه، والحفاظ على `assigned_at`	`UserController@updateRoles`
3	فحص الحظر قبل التحقق من كلمة المرور	أي شخص يعرف أن الحساب محظور بمجرد كتابة البريد (تسريب معلومات)	`Auth::validate()` أولًا ثم فحص الحظر، وإرجاع المستخدم مع roles	`AuthController@login`
4	المستخدم الجديد بلا دور	حساب بلا صلاحيات معروفة	إسناد الدور `user` تلقائيًا وحذف `Auth::login` غير اللازم	`AuthController@register`
5	`max:255` لاسم المستخدم	العمود `name` طوله 100، فالاسم الطويل يمر من التحقق ثم يفشل في MySQL	`max:100`	`register` · `updateProfile` · `updateUser`
6	لا حد لمحاولات الدخول	يسمح بتخمين كلمات المرور	`throttle:6,1` على login و register	`routes/api.php`
7	`/api/user` بدون الأدوار	الواجهة لا تعرف إن كان المستخدم أدمن	`$request->user()->load('roles')`	`routes/api.php`
8	أخطاء API ترجع HTML أو 500	طلب بدون توكن يحاول التحويل إلى `route('login')` غير الموجود	`shouldRenderJsonWhen` لكل مسارات `api/*`	`bootstrap/app.php`
9	الأدمن يستطيع حظر نفسه	يفقد الوصول إلى لوحة التحكم	منع حظر الحساب الشخصي	`UserController@banUser`
10	رسالة "حذف نهائي" مع حذف ناعم، والتوكنات تبقى	رسالة غير صحيحة، وجلسات المستخدم المحذوف تبقى صالحة	تصحيح الرسالة وحذف `tokens()`	`UserController@deleteUser`
11	import لكلاس غير موجود	`Illuminate\Support\Carbon` غير موجود وغير مستخدم	حذفه	`UserController.php`

3.2 الكتب

#	المشكلة	لماذا هي مشكلة	الإصلاح	المكان
12	توكن Sanctum حقيقي ملصق كتعليق	المستودع عام على GitHub فيستطيع أي شخص استخدامه	حُذف. يجب أيضًا حذفه من جدول `personal_access_tokens` إن كان موجودًا	آخر سطر في `BookController.php`
13	ملف PDF على القرص public	يمكن تنزيل الكتاب برابط مباشر بدون حساب	الحفظ على القرص الخاص `local` (`backend/storage/app/private`)	`BookController@store` · `updateBook`

#	المشكلة	لماذا هي مشكلة	الإصلاح	المكان
			vate)	
14	الملفات القديمة تبقى عند الاستبدال	التخزين يمتلئ بملفات غير مستخدمة	حذف القديم بعد نجاح `save()`	`BookController@updateBook`
15	الـ slug يتغير مع كل تعديل للعنوان	روابط صفحة الكتاب المحفوظة تنكسر	الـ slug يُنشأ مرة واحدة عند الإضافة	`BookController@updateBook`
16	إنشاء الكتاب وربط التصنيفات بدون Transaction	إن فشل الربط يبقى كتاب بلا تصنيفات	`DB::transaction` مع حذف الملفات المرفوعة عند الفشل	`BookController@store`

3.3 قاعدة البيانات والـ Seeders والبنية

#	المشكلة	لماذا هي مشكلة	الإصلاح	المكان
17	أسماء كلاسات لا تطابق أسماء الملفات (9 Controllers)	مخالفة PSR-4، فـ Composer يتجاهلها	تصحيح الأسماء، مثل `EditionController`	`app/Http/Controllers/*Controller.php`
18	`edition_assets.edition_id` مربوط بجدول `people`	علاقة خاطئة	ربطه بـ `editions` عبر Migration جديدة	`2026_11_07_000001_fix_catalog_schema_issues.php`
19	عمود `diaplay_order`	خطأ إملائي	إعادة التسمية إلى `display_order`	نفس الـ Migration
20	`publishers.website` إجباري	الناشر قد لا يملك موقعًا	`nullable`	نفس الـ Migration
21	كلمات مرور وبريد gmail داخل الـ Seeders	المستودع عام، فكلمات المرور مكشوفة	القيم من `backend/.env`، والحسابات التجريبية بنطاق example.com وفي بيئة local فقط	`AdminUserSeeder` · `UserSeeder`
22	أرقام `user_id` و `role_id` مكتوبة يدويًا	تذهب الأدوار لمستخدمين خطأ إن تغيّر الترتيب	البحث بالـ code والبريد، وكل مستخدم بلا دور يأخذ `user`	`UserRoleSeeder` · `DatabaseSeeder`
23	Docker لا يشغّل API قابلًا للوصول	لا Nginx، ولا composer install، وحد الرفع 2MB، ولا Volume ثابت، وكلمات مرور داخل الملف	إعادة إعداد كاملة (انظري `DOCKER_SETUP.md`)	`Dockerfile` · `docker-compose.yml` · `docker/`

لماذا Migration جديدة بدل تعديل القديمة؟ لأن الـ Migration التي نُفذت على قاعدة بيانات لا يُعاد تنفيذها عند تعديلها، فيصبح الكود والقاعدة مختلفين. القاعدة: الـ Migration المنفذة تاريخ لا يُعدَّل، والتصحيح دائمًا بـ Migration جديدة.

4. التوابع والراوتات الجديدة التي أُضيفت

هذه استكمالات لميزات كنتِ قد بدأتِها، أُضيفت حتى تعمل الواجهة.

4.1 الراوتات الجديدة أو المعدّلة

Method	URI	الحماية	التابع	ماذا يفعل
GET	`/books`	عام (كان للأدمن)	`BookController@index`	قائمة الكتب مع بحث `search`، وفلترة بالتصنيف `category` أو slug) `id`)، وترتيب `sort` (latest, oldest, title)، و `per_page` حتى 50. الافتراضي: الأحدث أولًا
GET	`/books/{slug}`	عام	`BookController@show`	تفاصيل كتاب واحد بالـ slug مع تصنيفاته، أو 404
GET	`/admin/books`	أدمن	`BookController@index`	نفس القائمة للوحة التحكم (لأن `/books` أصبح عامًا)
DELETE	`/books/{id}`	أدمن	`BookController@destroy`	حذف ناعم للكتاب، والملفات تبقى لأن الحذف قابل للاسترجاع
GET	`/categories`	عام (كان للمسجلين)	`CategoryController@index`	التصنيفات النشطة فقط مع `books_count`
GET	`/admin/categories`	أدمن	`CategoryController@adminIndex`	كل التصنيفات (نشطة وغير نشطة) مع عدد الكتب والتصنيف الأب
POST	`/admin/categories`	أدمن	`CategoryController@store`	إضافة تصنيف. الـ slug اختياري ويُولَّد فريدًا من الاسم
PATCH	`/admin/categories/{id}`	أدمن	`CategoryController@update`	تعديل التصنيف لا يمكن أن يكون التصنيف أبًا لنفسه
DELETE	`/admin/categories/{id}`	أدمن	`CategoryController@destroy`	حذف التصنيف. يُرفض بـ 422 إن كان مرتبطًا بكتب
GET	`/users`	أدمن (معدّل)	`UserController@displayusers`	أصبح بـ Pagination وبحث `search` وفلتر `status` (all, active, banned)، ويعيد roles و ban_info. المصفوفة الآن في `users.data`
POST	`/users/{id}/unban`	أدمن	`UserController@unbanUser`	رفع الحظر عن مستخدم
GET	`/roles`	أدمن	`RoleController@index`	قائمة الأدوار لاستخدامها في شاشة صلاحيات المستخدم

4.2 إضافات على الـ Models

المكان	الإضافة	ماذا تفعل	المفهوم
`Book`	`$hidden = ['file_path']`	مسار ملف الـ PDF الداخلي لا يُرسل في الـ API	إخفاء الحقول

المكان	الإضافة	ماذا تفعل	المفهوم
`Book`	`cover_url` و `has_ebook`	رابط كامل للغلاف، وهل توجد نسخة إلكترونية	Accessors بـ `Attribute::m` `ake` و `$appends`
`User`	`image_url`	رابط كامل للصورة الشخصية	نفس المفهوم
`Category`	`parent()` · `children()` · `books()`	علاقات التصنيف الأب والأبناء والكتب	علاقات self-referencing و many-to-many

5. جدول كل الراوتات الحالية

كل الراوتات تحت البادئة `/api`. مجموعة `user.active` تعني: مستخدم مسجل وغير محظور، و `admin.access` تعني: أدمن وغير محظور.

Method	URI	الحماية	التابع
POST	`/register`	عام + throttle	`AuthController@register`
POST	`/login`	عام + throttle	`AuthController@login`
GET	`/books`	عام	`BookController@index`
GET	`/books/{slug}`	عام	`BookController@show`
GET	`/categories`	عام	`CategoryController@index`
GET	`/user`	user.active	Closure (المستخدم مع roles)
POST	`/profile/update`	user.active	`UserController@updateProfile`
POST	`/logout`	user.active	`AuthController@logout`
POST	`/admin/reset-password`	admin.access	`AdminPasswordController@resetPassword`
GET	`/users`	admin.access	`UserController@displayusers`
GET	`/users/{id}`	admin.access	`UserController@showUser`
PATCH	`/users/{id}`	admin.access	`UserController@updateUser`
POST	`/users/{id}/ban`	admin.access	`UserController@banUser`
POST	`/users/{id}/unban`	admin.access	`UserController@unbanUser`
DELETE	`/users/{id}`	admin.access	`UserController@deleteUser`
PUT	`/users/{id}/roles`	admin.access	`UserController@updateRoles`
GET	`/roles`	admin.access	`RoleController@index`
GET	`/admin/books`	admin.access	`BookController@index`
POST	`/books`	admin.access	`BookController@store`
POST	`/books/{id}`	admin.access	`BookController@updateBook`
DELETE	`/books/{id}`	admin.access	`BookController@destroy`
GET	`/admin/categories`	admin.access	`CategoryController@adminIndex`
POST	`/admin/categories`	admin.access	`CategoryController@store`
PATCH	`/admin/categories/{id}`	admin.access	`CategoryController@update`
DELETE	`/admin/categories/{id}`	admin.access	`CategoryController@destroy`

6. الميزات المطلوب بناؤها

الواجهات لكل هذه الميزات **جاهزة ومربوطة** بالعقود أدناه. بمجرد أن تضيفي الـ Route ويعمل الـ Controller تتفعّل الواجهة تلقائيًا خلال دقيقة تقريبًا، بدون أي تعديل في الواجهة.

6.0 قواعد عامة قبل البدء

- كل المسارات تحت `/api`. الحماية بالمجموعات الموجودة `user.active` و `admin.access`.
- أخطاء التحقق بالشكل الافتراضي لـ Laravel (422 مع `errors`).
- القوائم تُعاد بـ `paginate()` أو بـ `JsonResource::collection()`، والواجهة تقبل الشكلين.
- الكائن الواحد: `{ data: {...} }`. الصور: رابط كامل عبر Accessor مثل `cover_url`.
- **أضيفي الـ Route فقط عندما يكون الـ Controller جاهزًا.** الواجهة تعتبر الميزة جاهزة بمجرد وجود المسار.
- اقتراح للتعلم: استخدمي في الميزات الجديدة `FormRequest` للتحقق، و `JsonResource` لشكل الاستجابة، و `Policy` للصلاحيات.

كيف تعرف الواجهة أن الميزة جاهزة؟ ترسل طلب GET بدون توكن على مسار فحص لكل ميزة:

الميزة	مسار الفحص	الميزة	مسار الفحص
المؤلفون	`GET /authors`	السلة	`GET /cart`
إدارة المؤلفين	`GET /admin/authors`	الطلبات	`GET /orders`
المتابعة	`GET /me/following`	إدارة الطلبات	`GET /admin/orders`
القراءة	`GET /books/__probe__/progress`	الإشعارات	`GET /notifications/unread-count`
المكتبة	`GET /me/library`	التوصيات	`GET /me/recommendations`
المفضلة	`GET /me/favorites`	الإحصائيات	`GET /admin/stats`
المراجعات	`GET /books/__probe__/reviews`	المتجر	`GET /store/products`
إدارة المراجعات	`GET /admin/reviews`	إدارة المنتجات	`GET /admin/products`

إذا أعاد Laravel `404` برسالة `The route ... could not be found` فالميزة غير مبنية. أي استجابة أخرى (200، 401، 403، 405...) تعني أنها جاهزة.

6.1 نسيت كلمة المرور

استعادة كلمة المرور عبر البريد. جدول `password_reset_tokens` موجود مسبقًا.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
POST	`/forgot-password`	عام + throttle	Body: `{ email }`. نفس الرد دائمًا `200 { message }` سواء وُجد البريد أم لا
POST	`/reset-password`	عام + throttle	Body: `{ token, email, password, password_confirmation }`

استخدمي `Password::sendResetLink` و `Password::reset`. رابط البريد يشير إلى الواجهة: `http://localhost:3000/ar/reset-password?token=...&email=...`. أضيفي Mailpit إلى Docker لرؤية الرسائل أثناء التطوير. (صفحة الواجهة لهذه الميزة ستُبنى عندما تبدئين بها.)

6.2 نموذج Editions والمؤلفون

الكتاب هو العمل الأدبي، والنسخة (Edition) شكل منه: إلكترونية (ebook) أو ورقية (paperback). الجداول موجودة: `editions` و `formats` و `languages` و `publishers` و `people` و `book_credits`.

الجداول المقترحة:

- Seeder لـ `ebook` :formats و `paperback`، وللغات: `ar` و `en`.
- علاقات: `Book hasMany editions`، و `Book belongsToMany people` عبر `book_credits` مع `withPiv ot('role','display_order')`، و `Edition belongsTo` كل من book و format و language و publisher.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/books/{slug}`	عام	أضيفي للاستجابة: `authors[]`، و `editions[]` (format, language, publisher, publication_date, page_count, product)، و `rating { average, count }`
GET	`/books?author=&language=&format=&min_price=&max_price=&min_rating=`	عام	فلاتر جديدة، وترتيب `sort=popular|top_rated|best_selling`. البحث يشمل اسم المؤلف واسم التصنيف
GET	`/authors`	عام	`?search=&page=`. كل عنصر: `{ id, name, slug, biography, image_url, books_count }`
GET	`/authors/{slug}`	عام	`{ data: author + categories[] }`، ومع التوكن يُضاف `viewer: { is_following }`
POST	`/admin/authors`	أدمن	multipart: `name`, `biography?`, `image?`
POST	`/admin/authors/{id}`	أدمن	تعديل المؤلف (POST من أجل الملفات)
DELETE	`/admin/authors/{id}`	أدمن	422 إذا كان مرتبطًا بكتب

إضافة الكتاب وتعديله يقبلان لاحقًا: `authors[]` و `language_id` و `publisher_id` و `publication_date` و `page_count`. انقلي ملف الـ PDF من `books.file_path` إلى النسخة الإلكترونية بـ Migration بيانات تحافظ على الملفات الحالية.

6.3 القراءة الإلكترونية وتقدم القراءة

قارئ PDF في الواجهة جاهز: تنقّل بين الصفحات والفصول، وتكبير، وأنماط قراءة، وحفظ آخر صفحة. المطلوب منك بث الملف وحفظ التقدم.

الجداول المقترحة:

- `reading_progress(user_id, book_id, page, total_pages, updated_at)` بمفتاح فريد على (user_id, book_id).

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/books/{slug}/read`	مستخدم	يبث الملف من القرص الخاص: `Storage::disk('local')->response($path)` مع `Content-Type: application/pdf`
GET	`/books/{slug}/progress`	مستخدم	`{ data: { page, total_pages, percent, updated_at } }` أو `{ data: null }`
PUT	`/books/{slug}/progress`	مستخدم	Body: `{ page, total_pages }`. التحقق: page بين 1 و total_pages

حفظ التقدم يجعل حالة الكتاب في المكتبة "أقرأه حاليًا" تلقائيًا، وعند الوصول للصفحة الأخيرة تصبح "أكملته" "لا يُرسل مسار الملف للواجهة أبدًا.

6.4 المكتبة الشخصية والمفضلة

رفوف القراءة (أريد قراءته، أقرأه، أكملته) وقائمة المفضلة.

الجداول المقترحة:

- `library_items(user_id, book_id, status, timestamps)` بفريد (user, book).
- `favorites(user_id, book_id, created_at)`.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/me/library`	مستخدم	`?status=want_to_read|reading|finished&page=`. كل عنصر: `{ book, status, added_at, progress? }`
PUT	`/me/library/{book_id}`	مستخدم	Body: `{ status }`. يضيف الكتاب أو يحدّث حالته
DELETE	`/me/library/{book_id}`	مستخدم	إزالة من المكتبة
GET	`/me/favorites`	مستخدم	Paginator لكتب المفضلة
PUT	`/me/favorites/{book_id}`	مستخدم	إضافة للمفضلة. تكرار الطلب لا يكرر الإضافة (Idempotent)
DELETE	`/me/favorites/{book_id}`	مستخدم	إزالة من المفضلة

مهم: في `GET /books/{slug}` أضيفي الحقل `viewer: { library_status, is_favorite, my_review_id }` عند وجود توكن صالح، و `null` للزائر. بهذا تعرض الواجهة الزر المناسب، مثل "إزالة من المفضلة" بدل "إضافة". لأن المسار عام، استخدمي داخله `auth('sanctum')->user()`.

6.5 التقييمات والمراجعات والبلاغات

تقييم من 1 إلى 5 مع مراجعة اختيارية، مراجعة واحدة لكل مستخدم لكل كتاب، وبلاغات يعالجها الأدمن.

الجداول المقترحة:

- `reviews(user_id, book_id, rating, body, timestamps)` بفريد (user, book).
- `review_reports(review_id, user_id, reason, note, status, timestamps)`.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/books/{slug}/reviews`	عام	`?sort=latest|top&page=`. كل عنصر: `{ id, rating, body, user{id,name,image_url}, created_at, updated_at }`
POST	`/books/{slug}/reviews`	مستخدم	Body: `{ rating: 1..5, body? }`. 422 إن كانت له مراجعة سابقة
PATCH	`/reviews/{id}`	المالك (Policy)	Body: `{ rating?, body? }`
DELETE	`/reviews/{id}`	المالك أو الأدمن	حذف المراجعة
POST	`/reviews/{id}/report`	مستخدم	Body: `{ reason: spam|offensive|spoiler|other, note? }`
GET	`/admin/reviews`	أدمن	`?reported=1&search=&page=` مع `book` و `reports_coun t`
GET	`/admin/reports`	أدمن	`?status=open|resolved`. كل عنصر: `{ id, reason, note, status, review, reporter, created_at }`
PATCH	`/admin/reports/{id}`	أدمن	Body: `{ status: "resolved", action: "dismiss" | "delete_review" }`

أضيفي `rating: { average, count }` إلى استجابة الكتاب (عبر `withAvg` و `withCount`).

6.6 المتجر والمخزون

المنتج هو نسخة ورقية قابلة للبيع.

الجداول المقترحة:

- `products(edition_id unique, price decimal(10,2), currency char(3), stock, shipping_info?, status active|hidden, timestamps)`.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/store/products`	عام	`?search=&category=&in_stock=1&page=`. كل عنصر: `{ id, price, currency, stock, in_stock, shipping_info, book{id,title,slug,cover_url,authors} }`
GET	`/admin/products`	أدمن	`?low_stock=1&page=`
POST	`/admin/products`	أدمن	Body: `{ edition_id, price, currency, stock, shipping_info?, status }`
PATCH	`/admin/products/{id}`	أدمن	تعديل السعر والمخزون والحالة ومعلومات الشحن

السعر يُرسل كنص عشري مثل "12.00" وليس رقمًا، لتجنب أخطاء الأعداد العشرية.

6.7 السلة

سلة المستخدم. كل الحسابات في الباك إند فقط، والواجهة لا تحسب أي سعر.

الجداول المقترحة:

- `cart_items(user_id, product_id, quantity, timestamps)`.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/cart`	مستخدم	`{ data: { items[], subtotal, shipping, total, currency, count } }`. كل عنصر: `{ id, quantity, unit_price, line_total, available, product{id,stock,book} }`
POST	`/cart/items`	مستخدم	Body: `{ product_id, quantity: 1..10 }`. إن كان المنتج موجودًا تُجمع الكمية
PATCH	`/cart/items/{id}`	مستخدم	Body: `{ quantity }`
DELETE	`/cart/items/{id}`	مستخدم	إزالة المنتج من السلة

قاعدة الشحن لم تُحدد بعد (سعر ثابت أم حسب المدينة). ضعيها مؤقتًا في `config/store.php`.

6.8 الطلبات والدفع عند الاستلام

إنشاء طلب من السلة ومتابعته. الدفع عند الاستلام فقط، ولا تُخزن أي بيانات بطاقات.

الجداول المقترحة:

- `orders(user_id, number, status, payment_method, subtotal, shipping, total, currency, name, phone, city, address, notes, timestamps)`.
- `order_items(order_id, product_id, title, quantity, unit_price, line_total)`، مع نسخ السعر وقت الشراء.
- `order_status_history(order_id, status, changed_by, created_at)`.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
POST	`/orders`	مستخدم	Body: `{ name, phone, city, address, notes? }`. ينشئ الطلب من السلة ويفرّغها، ويعيد `{ data: order }`
GET	`/orders`	مستخدم	طلباتي (Paginator)
GET	`/orders/{id}`	المالك	`{ data: { id, number, status, payment_method, items[], subtotal, shipping, total, currency, shipping_address, history[], created_at } }`
POST	`/orders/{id}/cancel`	المالك	فقط إذا كانت الحالة pending
GET	`/admin/orders`	أدمن	`?status=&search=&page=` مع `user`
GET	`/admin/orders/{id}`	أدمن	تفاصيل الطلب مع `user`
PATCH	`/admin/orders/{id}/`	أدمن	Body: `{ status }`. 422 لأي انتقال غير مسموح

Method	URI	الحماية	الطلب والاستجابة

الحالات: `delivered` ← `shipped` ← `preparing` ← `confirmed` ← `pending`، ويمكن الانتقال إلى `cancelled` قبل الشحن فقط.

إنشاء الطلب داخل `DB::transaction` مع `lockForUpdate` على المخزون: تحقّق، ثم إنقاص، ثم إنشاء، ثم تفريغ السلة.

6.9 بيانات التواصل والعنوان

إكمال الملف الشخصي (موجود جزئيًا).

الجداول المقترحة:

- Migration جديدة تضيف `phone` و `city` و `address` (في users أو user_profiles).

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
POST	`/profile/update`	مستخدم	أضيفي: `display_name?`, `bio?`, `phone?`, `city?`, `address?`
GET	`/user`	مستخدم	يعيد أيضًا `profile`

6.10 متابعة المؤلفين والإشعارات

متابعة المؤلفين، وإشعارات قاعدة البيانات.

الجداول المقترحة:

- `author_follows(user_id, person_id, created_at)`.
- جدول `notifications` الافتراضي: `php artisan make:notifications-table`.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
PUT	`/authors/{id}/follow`	مستخدم	متابعة
DELETE	`/authors/{id}/follow`	مستخدم	إلغاء المتابعة
GET	`/me/following`	مستخدم	المؤلفون الذين أتابعهم
GET	`/notifications`	مستخدم	`?unread=1&page=`. كل عنصر: `{ id, type, data{type,title,body,url}, read_at, created_at }`
GET	`/notifications/unread-count`	مستخدم	`{ count }`
POST	`/notifications/{id}/read`	مستخدم	تعليم إشعار كمقروء
POST	`/notifications/read-all`	مستخدم	تعليم الكل كمقروء

الأنواع: `new_book` و `author_new_book` و `book_updated` و `order_status_changed` و `back_in_stock` و `promotion`. الحقل `url` مسار داخل الواجهة بدون اللغة، مثل `/books/slug`. الإرسال لعدد كبير من المتابعين يكون عبر Queue (Job).

6.11 التوصيات والكتب المشابهة

نسخة أولى بسيطة: التصنيفات والمؤلفون الأكثر تكرارًا في مكتبة المستخدم وتقييماته العالية.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/me/recommendations`	مستخدم	`{ data: { for_reading: Book[], for_buying: StoreProduct[] } }`
GET	`/books/{slug}/similar`	عام	حتى 10 كتب بنفس التصنيفات أو المؤلف، دون الكتاب نفسه

6.12 إحصائيات لوحة التحكم

الأعداد الأساسية تعمل حاليًا من الراوتات الموجودة. المطلوب الإحصائيات الإضافية.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
GET	`/admin/stats`	أدمن	`{ data: { books, users, users_banned, orders{total,pending,revenue,currency}, reading{active_readers_30d}, reviews{total,reported_open}, top_books[], low_stock[] } }`

6.13 ترجمة رسائل الـ API

رسائل التحقق حاليًا إنجليزية، ورسائل الـ Controllers عربية فقط.

الـ Endpoints:

Method	URI	الحماية	الطلب والاستجابة
—	Middleware	—	يقرأ `Accept-Language: ar|en` ويضبط `app()->setLocale()` ثم `php artisan lang:publish` وإضافة `lang/ar/validation.php`

7. قائمة المتابعة وخطة بناء المشروع

✔ منجز · ◗ منجز جزئيًا · ☐ متبقٍ

اطبعي هذه القائمة أو انسخيها، وضعي إشارة على كل بند تنهينه. الواجهة لكل بنود الباك إند المتبقية جاهزة، إلا ما ذُكر في عمود الملاحظات.

7.1 خطة المراحل

الحالة	البند	المسؤول	ملاحظات
✔ منجز	المرحلة A: دراسة المشروع ومراجعة الكود	Dima	تقرير Discovery
✔ منجز	المرحلة B: إصلاح الأخطاء الأمنية والبرمجية	Dima	القسم 3 من هذا الدليل
✔ منجز	المرحلة C: بيئة Docker كاملة	Dima	app, nginx, db, frontend
✔ منجز	المرحلة D: الهوية البصرية والـ Design System	Dima	عربي وإنجليزي، RTL/LTR
✔ منجز	المرحلة E: واجهات الميزات الجاهزة في الباك إند	Dima	الكتالوج، الحساب، لوحة التحكم
✔ منجز	المرحلة F: واجهات الميزات بانتظار الباك إند + العقود	Dima	القسم 6
☐ متبقٍ	المرحلة G: بناء الميزات المتبقية في الباك إند	صديقتي	بالترتيب في 7.3
☐ متبقٍ	المرحلة H: الربط الحقيقي ومراجعة الكود معًا وتحسينه	معًا	بعد كل ميزة
☐ متبقٍ	المرحلة I: اختبارات (Feature Tests) لكل Endpoint	صديقتي	`php artisan make:test`
☐ متبقٍ	المرحلة J: إعداد الإنتاج والنشر	معًا	SHOW_API_CONTRACTS=false، HTTPS، config:cache

7.2 الباك إند: ما أُنجز

الحالة	البند	المسؤول	ملاحظات
✔ منجز	التسجيل وتسجيل الدخول والخروج (Sanctum)	صديقتي + إصلاحات	
✔ منجز	تحديد محاولات الدخول (throttle)	Dima	
✔ منجز	الملف الشخصي: الاسم والصورة وكلمة المرور	صديقتي	
✔ منجز	الحظر المؤقت والدائم + رفع الحظر	صديقتي + Dima	
✔ منجز	إدارة المستخدمين للأدمن (عرض، تعديل، حذف، أدوار، كلمة مرور)	صديقتي + إصلاحات	
✔ منجز	قائمة الأدوار	Dima	
✔ منجز	قائمة الكتب العامة مع البحث والفلترة والترتيب	صديقتي + Dima	
✔ منجز	عرض كتاب واحد	Dima	
✔ منجز	إضافة وتعديل وحذف الكتب مع رفع الغلاف والـ PDF	صديقتي + Dima	الملف على قرص خاص

الحالة	البند	المسؤول	ملاحظات
✔ منجز	التصنيفات العامة + إدارتها للأدمن	صديقي + Dima	
✔ منجز	Seeders آمنة + Migration تصحيحية	Dima	

7.3 الباك إند: المطلوب منك (بالترتيب المقترح)

الحالة	البند	المسؤول	ملاحظات
☐ متبقٍ	6.1 نسيت كلمة المرور	صديقي	صفحة الواجهة تُبنى عند بدئك
☐ متبقٍ	6.2 Seeders لـ formats و languages	صديقي	
☐ متبقٍ	6.2 علاقات Edition و Book و Person	صديقي	
☐ متبقٍ	6.2 نقل ملف PDF إلى النسخة الإلكترونية	صديقي	
☐ متبقٍ	6.2 authors و editions و rating في استجابة الكتاب	صديقي	
☐ متبقٍ	6.2 فلاتر الكتب الإضافية	صديقي	الواجهة تحتاج إضافة الفلاتر الجديدة
☐ متبقٍ	6.2 المؤلفون: القائمة والتفاصيل	صديقي	
☐ متبقٍ	6.2 إدارة المؤلفين للأدمن	صديقي	
☐ متبقٍ	6.2 ربط المؤلفين واللغة والنسخ في إضافة الكتاب	صديقي	الواجهة تحتاج حقولًا جديدة في نموذج الكتاب
☐ متبقٍ	6.3 بث ملف الـ PDF	صديقي	
☐ متبقٍ	6.3 حفظ وقراءة تقدم القراءة	صديقي	
☐ متبقٍ	6.4 المكتبة الشخصية (الرفوف)	صديقي	
☐ متبقٍ	6.4 المفضلة	صديقي	
☐ متبقٍ	6.4 حقل viewer في صفحة الكتاب	صديقي	
☐ متبقٍ	6.5 المراجعات (إضافة، تعديل، حذف، قائمة)	صديقي	
☐ متبقٍ	6.5 البلاغات وإدارتها	صديقي	
☐ متبقٍ	6.6 المنتجات والمخزون	صديقي	الواجهة تحتاج شاشة إنشاء منتج من صفحة الكتاب
☐ متبقٍ	6.7 السلة	صديقي	
☐ متبقٍ	6.8 الطلبات (إنشاء، عرض، إلغاء)	صديقي	
☐ متبقٍ	6.8 إدارة الطلبات وتحديث الحالة	صديقي	
☐ متبقٍ	6.9 الهاتف والعنوان في الملف الشخصي	صديقي	الواجهة تحتاج الحقول في الملف الشخصي
☐ متبقٍ	6.10 متابعة المؤلفين	صديقي	
☐ متبقٍ	6.10 الإشعارات + Queue Worker	صديقي	
☐ متبقٍ	6.11 التوصيات والكتب المشابهة	صديقي	
☐ متبقٍ	6.12 إحصائيات لوحة التحكم	صديقي	

الحالة	البند	المسؤول	ملاحظات
☐ متبقٍ	6.13 ترجمة رسائل الـ API	صديقتي	

7.4 الواجهة

الحالة	البند	المسؤول	ملاحظات
✔ منجز	الهوية البصرية، الخطوط، العربية والإنجليزية، الموبايل	Dima	
✔ منجز	الرئيسية، تصفح الكتب، صفحة الكتاب، التصنيفات	Dima	مربوطة
✔ منجز	تسجيل الدخول، التسجيل، الملف الشخصي	Dima	مربوطة
✔ منجز	لوحة التحكم: الكتب، التصنيفات، المستخدمون	Dima	مربوطة
✔ منجز	المؤلفون + إدارة المؤلفين	Dima	بانتظار 6.2
✔ منجز	قارئ PDF مع حفظ التقدم	Dima	بانتظار 6.3
✔ منجز	مكتبتي، المفضلة، أزرار صفحة الكتاب	Dima	بانتظار 6.4
✔ منجز	المراجعات + إدارة البلاغات	Dima	بانتظار 6.5
✔ منجز	المتجر، السلة، إتمام الطلب، طلباتي	Dima	بانتظار 6.6 إلى 6.8
✔ منجز	إدارة الطلبات والمنتجات	Dima	بانتظار 6.6 و 6.8
✔ منجز	الإشعارات وعدّادات الهيدر	Dima	بانتظار 6.10
✔ منجز	التوصيات والإحصائيات	Dima	بانتظار 6.11 و 6.12
☐ متبقٍ	صفحة إعادة تعيين كلمة المرور	Dima	بعد 6.1
☐ متبقٍ	حقول المؤلفين واللغة والنسخ في نموذج الكتاب	Dima	بعد 6.2
☐ متبقٍ	فلاتر الكتب الإضافية	Dima	بعد 6.2
☐ متبقٍ	إنشاء منتج من صفحة الكتاب	Dima	بعد 6.6
☐ متبقٍ	الهاتف والعنوان في الملف الشخصي وتعبئة نموذج الطلب تلقائيًا	Dima	بعد 6.9
☐ متبقٍ	صفحة كل المراجعات مع التقسيم إلى صفحات	Dima	بعد 6.5

بعد إنهاء كل ميزة: أخبريني، نختبر الربط الحقيقي معًا، ثم نراجع الكود ونحسّنه (Form Requests، API Resources، Policies). هكذا تتعلمين من كل خطوة.